

5 Ausnahmebehandlung

```
try {  
    ...  
} catch (...) {  
    ...  
} catch (...) {  
    ...  
} finally {  
    ...  
}
```

- Robustheit hat bei Java oberste Priorität
- Programme sollen auch in Ausnahme- oder Fehlersituationen nicht abstürzen sondern sicher weiterlaufen
- typische Ausnahmesituationen:
 - Division durch Null
 - Lese/Schreibfehler bei Dateien
 - etc.
- Ausnahmesituationen können oft nicht dort behandelt werden, wo sie auftreten
- bei Auftreten von Ausnahmesituationen wird ein Ausnahmeobjekt **erzeugt** und signalisiert (**geworfen**), auf das **reagiert** werden kann
 - Ausnahmeobjekte werden weitergereicht, bis darauf reagiert wird
 - das Programm wird abgebrochen, wenn auf eine Ausnahmesituation nicht reagiert wird

Eigenschaften von Algorithmen (PRO1):

Ausführbarkeit der Aktionen

- vom Ausführenden darf **nichts Unmögliches** verlangt werden
- der Algorithmus darf **nicht** in einen **undefinierten Zustand** geraten, z.B. bei Division durch 0

Beispiel: Programmabbruch, wenn auf eine Ausnahmesituation nicht reagiert wird

```
public class Motivation {  
    public static void main(String[] args) {  
        Motivation m = new Motivation();  
9:        m.a();  
    }  
    public void a() {  
        IO.println(">a");  
14:        b();  
        IO.println("<a");  
    }  
    public void b() {  
        IO.println(">b");  
20:        c();  
        IO.println("<b");  
    }  
    public void c() {  
        IO.println(">c");  
26:        IO.println(42 / 0);  
        IO.println("<c");  
    }  
}
```

```
java Motivation
```

```
>a  
>b  
>c  
Exception in thread "main"  
java.lang.ArithmeticException: / by zero  
    at Motivation.c(Motivation.java:26)  
    at Motivation.b(Motivation.java:20)  
    at Motivation.a(Motivation.java:14)  
    at Motivation.main(Motivation.java:9)
```

RuntimeExceptions (z.B. Division durch Null)

- signalisieren Fehler, die zur Übersetzungszeit nicht vorhersehbar sind (Programmierfehler im Quelltext)
- müssen nicht explizit angegeben werden, können aber wie andere Ausnahmen behandelt werden

Exceptions (z.B. Lese/Schreibfehler bei Dateien)

- signalisieren "leichte", d.h. behebbare Fehler
- müssen explizit angegeben oder behandelt werden; dies wird vom Compiler überprüft

Errors (z.B. Überlauf am Laufzeitkeller)

- signalisieren "schwere", d.h. nicht behebbare Fehler
- nach Auftreten eines Errors ist das Fortsetzen des Programms meist nicht sinnvoll

RuntimeExceptions, Exceptions und Errors sind Objekte bestimmter Klassen (die von einer Klasse Throwable *abgeleitet* sind, siehe Vererbung)

Es gibt bereits eine Reihe von vordefinierten Klassen für Ausnahmen:

NullPointerException	String s = null; int k = s.length();
IndexOutOfBoundsException	a[-1]
NegativeArraySizeException	new int[-1]
ArithmeticException	int i = 3; int j = i / 0;
NumberFormatException	Integer.parseInt("12x45")
MissingFormatArgumentException	String.format("i=%d,j=%d", i)
FileNotFoundException	new FileInputStream("readme.txt")
AssertionError	assert false;
StackOverflowError	void r() { r(); }

werden von der
Java VM ausgeführt

Konstruktoren und Methoden (Auszug) am Beispiel für die Klasse NumberFormatException:

<code>NumberFormatException()</code>	
<code>NumberFormatException(String msg)</code>	
<code>String getMessage()</code>	liefert gespeicherte Fehlermeldung
<code>String toString()</code>	liefert Art der Ausnahme und gespeicherte F.
<code>void printStackTrace()</code>	gibt Methodenaufkette aus

For input string: "12x45"
`java.lang.NumberFormatException: For input string: "12x45"`

`java.lang.NumberFormatException: For input string: "12x45"`
 at java.base/java.lang.NumberFormatException.forInputString(NumberFormatException.java:67)
 at java.base/java.lang.Integer.parseInt(Integer.java:662)
 at java.base/java.lang.Integer.parseInt(Integer.java:778)
 at ch8.Exceptions.printNumberFormatException(Exceptions.java:89)
 at ch8.Exceptions.main(Exceptions.java:113)

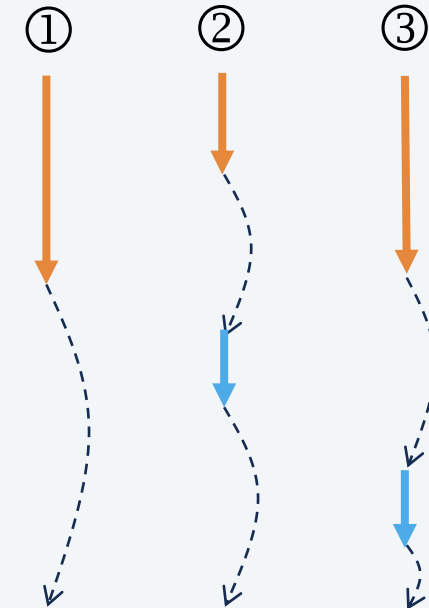
Damit nach Auftreten einer Ausnahmesituation ein Programm sinnvoll und definiert weiterlaufen kann, wird die try-Anweisung eingesetzt

geschützter Block mit
Anweisungen, die
Ausnahme auslösen
können

Ausnahmebehandlung
(beliebig viele)

```
try {  
    Anweisung1  
    Anweisung2  
    ...  
    Anweisungn  
} catch (Ausnahmetyp1 e) {  
    Anweisungenc1  
} catch (Ausnahmetyp2 e) {  
    Anweisungenc2  
}
```

- es muss nicht für alle Ausnahmetypen eine Ausnahmebehandlung vorgesehen sein
- nicht behandelte Ausnahmeobjekte werden an den Aufrufer weitergeleitet

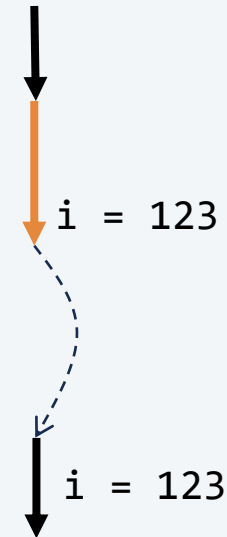


1. Ausführung ohne Ausnahme
2. Ausnahmesituation in Anweisung₂ (Ausnahmetyp₁)
3. Ausnahmesituation in Anweisung_n (Ausnahmetyp₂)

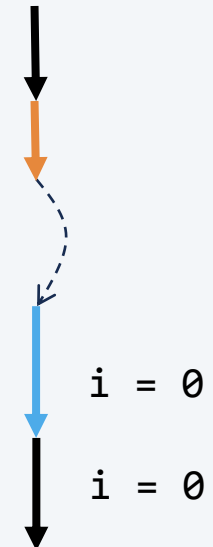
Beispiel

```
String s = ... ;  
int i;  
try {  
    i = Integer.parseInt(s);  
    IO.println("i = " + i);  
} catch (NumberFormatException e) {  
    i = 0;  
    IO.println("i = " + i);  
}  
IO.println("i = " + i);
```

s = "123";



s = "12x3";



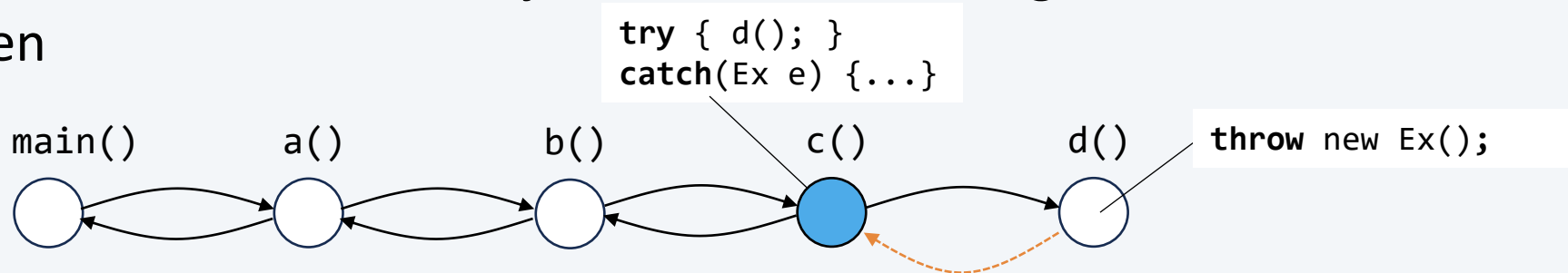
throw Ausdruck;

- signalisiert („wirft“) ein Ausnahmeobjekt mit entsprechenden Fehlerinformationen
- Ausdruck muss Ausnahmeobjekt liefern (z.B. `new NumberFormatException()`)

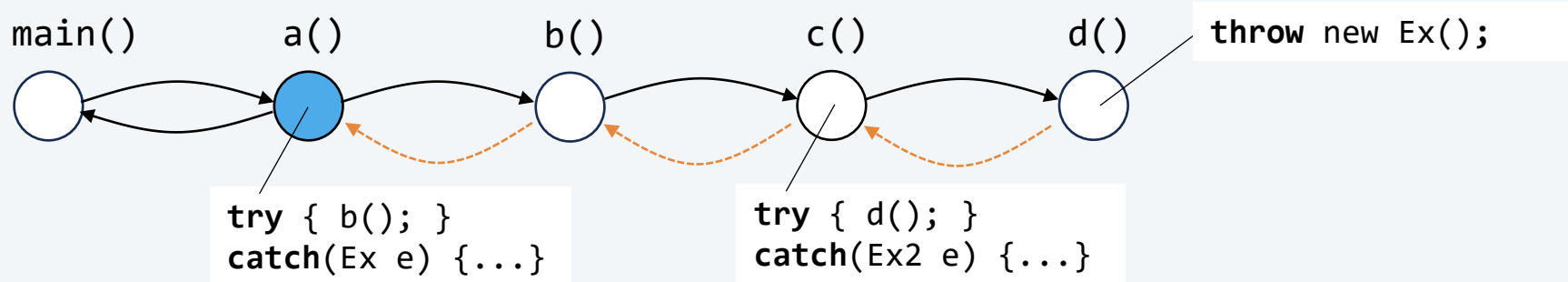
```
public static int parseInt(String s) {  
    if (s == null) throw new NumberFormatException("null string");  
    int value = 0;  
    for (int i = 0; i < s.length(); i++) {  
        char ch = s.charAt(i);  
        if ('0' <= ch && ch <= '9') {  
            value = 10 * value + (ch - '0');  
        } else {  
            throw new NumberFormatException(s);  
        }  
    }  
    return value;  
}
```

Hinweis: Eine Exception wird nicht bereits durch das Erzeugen eines neuen Objekts einer Ausnahmeklasse ausgelöst, sondern erst durch eine throw-Anweisung.

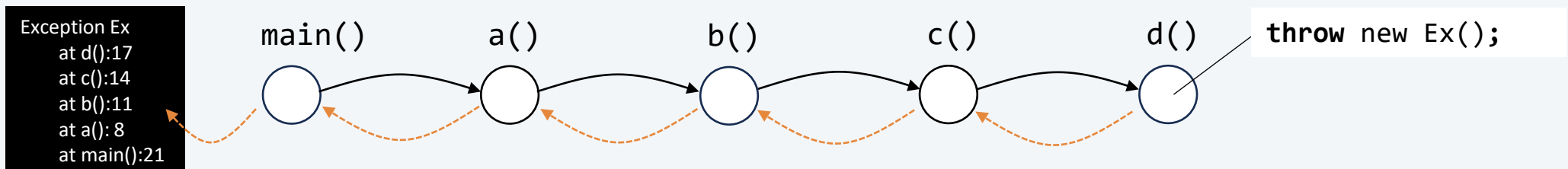
Methoden können Ausnahmeobjekte entweder abfangen oder nach außen weiterleiten



Nicht behandelte Ausnahmeobjekte werden an den Aufrufer weitergeleitet



Wird eine Ausnahme nirgendwo behandelt, so wird die Ausführung des Programms abgebrochen



Throwable

Error

- AssertionError
- VirtualMachineError
 - StackOverflowError
 - ...

Schwere, d.h. nicht behebbare Fehler

- nach Auftreten eines Errors ist das Fortsetzen des Programms meist nicht sinnvoll

Exception

RuntimeException

- NullPointerException
- IndexOutOfBoundsException
- ArithmeticException
- IllegalArgumentException
 - NumberFormatException
 - ...

Fehler, die zur Übersetzungszeit nicht vorhersehbar sind

- Programmierfehler im Quelltext
- falsche Verwendung von Methoden
- ...

IOException

- EOFException
- FileNotFoundException
- ...

Leichte, d.h. behebbare Fehler

- müssen **explizit angegeben oder behandelt** werden (geprüfte Ausnahmen, *checked exceptions*)
- dies wird vom Compiler überprüft

InterruptedException

...

- für Ausnahmesituationen können von Entwickler*innen selbst Exceptions definiert werden
- dazu muss eine (direkte oder indirekte) Unterklasse von Exception implementiert werden

```
public class MyException extends Exception
{
    ...
}
```

- benutzerdefinierte Ausnahmen werden mit der throw-Anweisung unter benutzerdefinierten Bedingungen ausgelöst
- die neue Ausnahmeklasse sollte zumindest zwei Konstruktoren haben für Objekte mit und ohne Fehlermeldung
- weitere Datenkomponenten und Methoden können je nach Bedarf verwendet werden

Beispiel: Version 1 mit vordefinierter Ausnahme

```
public static int parse(String digits, int base) {  
    int value = 0;  
    for (int i = 0; i < digits.length(); i++) {  
        char ch = digits.charAt(i);  
        if (ch < '0' || ch > '9') {  
            throw new NumberFormatException(  
                "Invalid character '%c' at position %d".formatted(ch, i));  
        }  
        int digit = ch - '0';  
        if (digit >= base) {  
            throw new NumberFormatException(  
                "Invalid character '%c' at position %d".formatted(ch, i));  
        }  
        value = value * base + digit;  
    }  
    return value;  
}
```

```
String digits = scanner.nextLine();  
try {  
    int value = Num.parse(digits, 2);  
    IO.print("The value of ...");  
} catch (NumberFormatException e) {  
    IO.println("Error: " + e.getMessage());  
}
```

Beispiel: Version 2 mit benutzerdefinierter Ausnahme

```
public static int parse(String digits, int base) {
    int value = 0;
    for (int i = 0; i < digits.length(); i++) {
        char ch = digits.charAt(i);
        if (ch < '0' || ch > '9') {
            throw new InvalidDigitException("invalid digit", i, value);
        }
        int digit = ch - '0';
        if (digit >= base) {
            throw new InvalidDigitException(
                "Invalid digit %d in base %d".formatted(digit, base), i, value);
        }
        value = value * base + digit;
    }
    return value;
}
```

```
Enter a binary number: 101x10
Error: Invalid digit
101x10
  ^
Partial value before error: 5
```

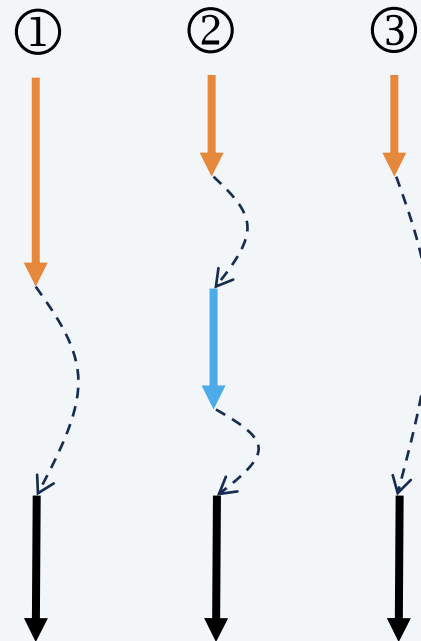
```
String digits = scanner.nextLine();
try {
    int value = Num.parse(digits, 2);
    IO.format("The value of ...");
} catch (InvalidDigitException e) {
    IO.println("Error: " + e.getMessage());
    for (int i = 0; i < e.getPosition(); i++) {
        IO.print(" ");
    }
    IO.println("^");
    IO.println("Partial ..." + e.getPartialValue());
}
```

Beispiel: Version 2 mit benutzerdefinierter Ausnahme

```
public class InvalidDigitException extends NumberFormatException {  
  
    private final int position;  
    private final int partialValue;  
  
    public InvalidDigitException() {  
        this(null, 0, 0);  
    }  
    public InvalidDigitException(String msg) {  
        this(msg, 0, 0);  
    }  
    public InvalidDigitException(String message, int position, int partialValue) {  
        super(message);  
        this.position = position;  
        this.partialValue = partialValue;  
    }  
    public int getPosition() {  
        return position;  
    }  
    public int getPartialValue() {  
        return partialValue;  
    }  
}
```

- Abschlussarbeiten werden im **optionalen** finally-Block zusammengefasst
- dieser Block wird immer ausgeführt, unabhängig davon, ob eine Exception aufgetreten ist, auch dann, wenn keine oder kein passender catch-Block vorhanden ist

```
try {  
    Anweisung1  
    ...  
    Anweisungn  
} catch (Ausnahmetyp1 e) {  
    Anweisungenc1  
} catch (Ausnahmetyp2 e) {  
    Anweisungenc2  
} finally {  
    Anweisungenf  
}
```



1. Ausführung ohne Ausnahme
2. Ausnahmesituation in Anweisung₂ (Ausnahmetyp₁)
3. Ausnahmesituation in Anweisung₂ mit Ausnahmetyp, der hier nicht behandelt wird

Beispiel: für ein Scanner-Objekt muss die Methode `close()` aufgerufen werden

Version 1

```
Scanner s = new Scanner(System.in);
IO.print("a = ");
int a = s.nextInt();
IO.println("a*a = "+ (a * a));
```

Compiler warning:

Resource leak: 's' is never closed

Version 2

```
Scanner s = new Scanner(System.in);
IO.print("a = ");
int a = s.nextInt();
IO.println("a*a = "+ (a * a));
s.close();
```

Die Methode `close()` wird nicht aufgerufen, wenn in der Methode `nextInt` eine Ausnahme ausgelöst wird

Version 3

```
Scanner s = new Scanner(System.in);
IO.print("a = ");
try {
    int a = s.nextInt();
    IO.println("a*a = "+ (a * a));
} catch (InputMismatchException e) {
    ...
} finally {
    s.close();
}
```

Die Methode `close()` wird auf jeden Fall geschlossen

```
void m() throws Ausnahmetyp;
```

Exceptions ... geprüfte Ausnahmen (*checked exceptions*)

- nicht abgefangene Ausnahmen **müssen** bei der Methodendeklaration angegeben werden

```
void a() {  
    try {  
        b();  
    } catch (Ex1 e) {  
        ...  
    }  
}
```

```
void b() throws Ex1 {  
    try {  
        c();  
    } catch (Ex2 e) {  
        ...  
    }  
}
```

```
void c() throws Ex1, Ex2 {  
    ... throw new Ex1(); ...  
    ...  
    ... throw new Ex2(); ...  
}
```

- der Compiler weiß dadurch, dass `b()` eine `Ex1`-Ausnahme und `c()` eine `Ex1`- oder `Ex2`-Ausnahme auslösen kann
- der Compiler prüft, dass Ausnahmen abgefangen (`catch`) oder weitergeleitet (`throws`) werden